Aula 19 - Framework Web (Flask) e Rotas

Descrição: Introdução ao Servidor Web Python usando o microframework Flask. Desmistificação do roteamento web (@app.route), mapeamento de endpoints e renderização de arquivos HTML locais. Configuração do ambiente localhost (porta 5000) e modo de depuração para testes no ciclo de desenvolvimento.

Cenário Real: Você já parou para pensar que a internet, no seu núcleo, é apenas uma **gigantesca troca de mensagens**? Quando você digita "netflix.com" ou abre o aplicativo do seu banco, seu dispositivo atua como um ***cliente*** pedindo algo, e um computador robusto do outro lado do mundo, chamado ***servidor***, processa a regra de negócio e responde enviando telas, vídeos ou dados. No mercado de desenvolvimento atual, dominar o *back-end* – o "cérebro" que roda nesse servidor – é uma das habilidades mais requisitadas. Hoje, nós vamos transformar o seu próprio computador nesse cérebro. Usaremos Python junto com o **Flask**, um *microframework* de mercado escolhido por sua arquitetura minimalista (conhecida como *no-frills*), tão poderosa e veloz que é utilizada para levantar microsserviços na **Netflix** e no **Reddit**.

Pergunta: Se os nossos algoritmos em Python sempre rodaram isolados e fechados no terminal da nossa IDE, como nós fazemos para criar um "canal" e permitir que o navegador web consiga enviar requisições e receber respostas do nosso código?

Desenvolvimento e Prática

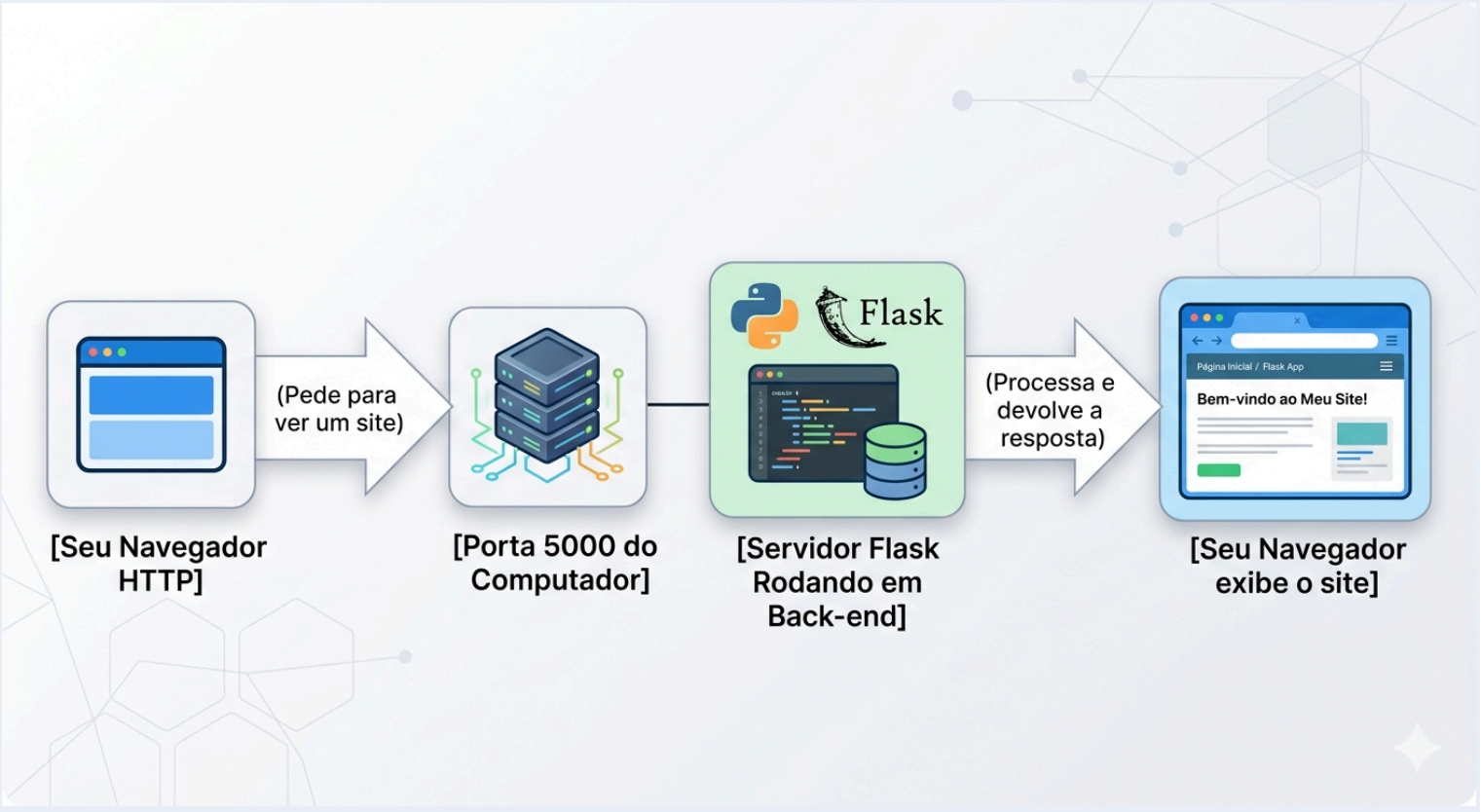
Nós vamos estruturar esta aula em três grandes passos. O foco é entendermos "por que" as coisas acontecem antes de apenas digitar o código. Se tiverem dúvidas, usem ferramentas de IA Generativa de forma ética: peçam para a IA explicar um conceito sob uma nova ótica ou analogia, nunca apenas para copiar a resposta pronta. O conhecimento real vem da prática!

1º Servidor Web e o Microframework Flask

A. Conceito: O Flask é classificado como um *microframework* web escrito em Python. Ele recebe o prefixo "micro" não por ser fraco, mas por ser **modular** e **minimalista**: ele não traz dezenas de bibliotecas pesadas e desnecessárias instaladas por padrão, permitindo que nós, desenvolvedores, tenhamos total controle para adicionar apenas o que o projeto precisar. Ele constrói nossa base em cima de um kit de ferramentas poderoso chamado ***Werkzeug*** (que lida com as requisições web) e o ***Jinja2*** (motor de templates).

No ciclo de desenvolvimento que faremos, o nosso próprio computador atuará como servidor local. A isso damos o nome de **localhost**, cujo endereço de IP interno normalmente é `127.0.0.1`. Esse ambiente isolado é perfeito porque impede acesso externo durante nossos testes e garante que nossa aplicação "ouça" o navegador através de uma "porta" específica do sistema, que por convenção no Flask é a porta `5000`.

B. Visualizando: Fluxo do Servidor Local:



C. Exemplo de Código:

```python
# Importamos a classe principal do microframework
from flask import Flask

# Inicializamos a nossa aplicação web (instanciando o objeto)
app = Flask(__name__)

# Garantimos que o servidor só suba se executarmos este arquivo diretamente
if __name__ == '__main__':
    # Ligamos o servidor no localhost, porta padrão, com modo de depuração ativado
    app.run(debug=True)
```

D. Explicação linha por linha:

- `from flask import Flask`: Trazemos a "caixa de ferramentas" do Flask para o nosso projeto.
- `app = Flask(__name__)`: Criamos uma variável chamada `app` que representa a nossa aplicação inteira. O `__name__` indica ao Flask onde ele deve procurar os recursos e arquivos do projeto.
- `if __name__ == '__main__':`: Boa prática de segurança e organização em Python para garantir que o *script* não vai tentar subir o servidor se for apenas importado por outro arquivo.
- `app.run(debug=True)`: A mágica acontece! Inicia o servidor local. O `debug=True` ativa o Modo de Depuração, que exibe erros detalhados (*tracebacks*) no navegador e reinicia o servidor automaticamente quando nós salvamos o arquivo, acelerando nossa produtividade.

E. Pesquise e Responda:

1. Qual o comando via terminal para instalar o Flask no ambiente virtual do seu projeto?
2. Defina com suas palavras a diferença entre um framework *full-stack* (como Django) e um *microframework* (como Flask).
3. O que representa o IP `127.0.0.1`?

4. Crie o arquivo `app.py` na sua IDE (VS Code), transcreva o código exemplo e execute. O que aparece no terminal?
5. Qual é a utilidade prática do parâmetro `debug=True` no nosso dia a dia de código?

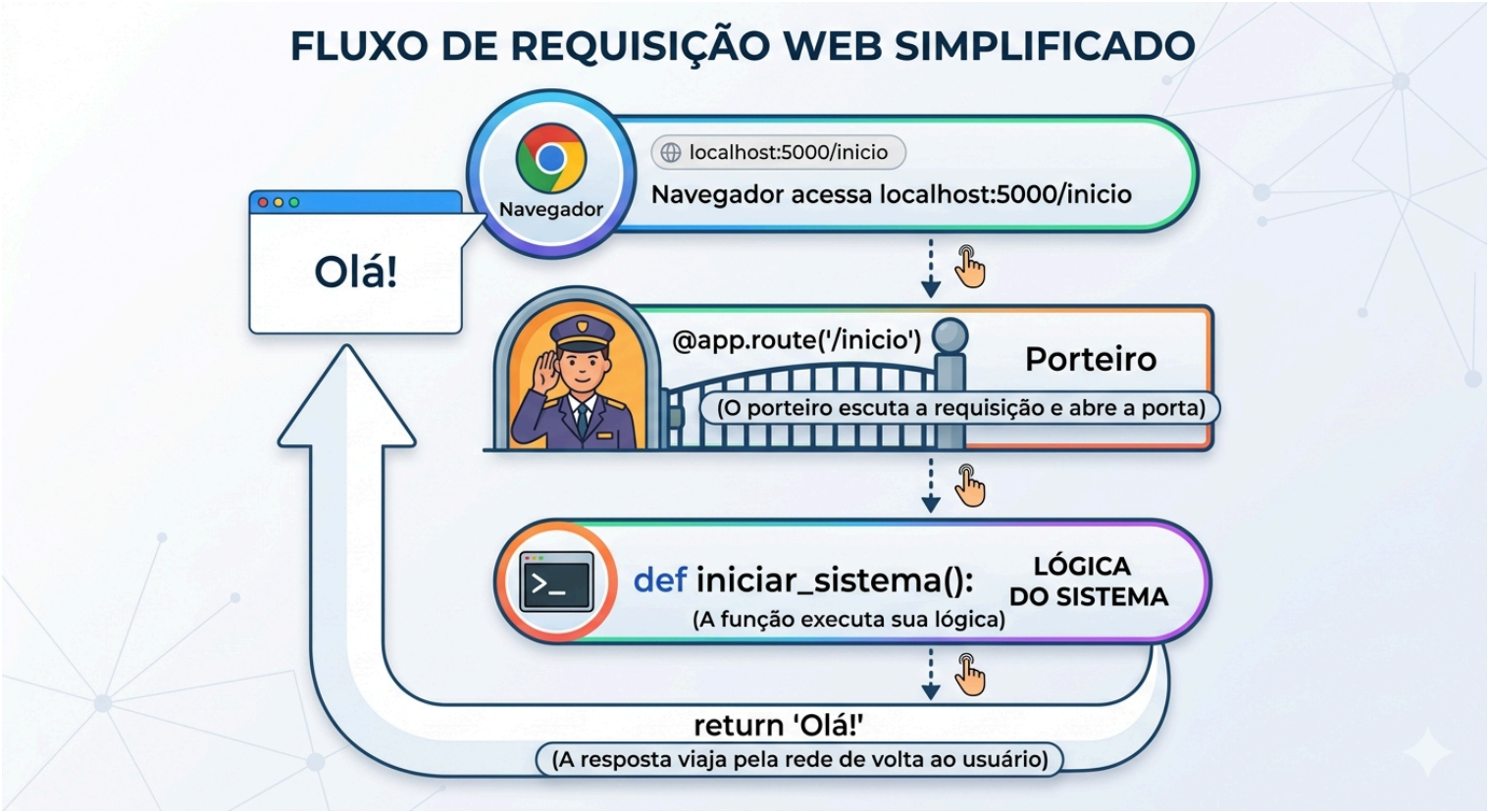
---

2. A Anatomia das Rotas (@app.route)

A. Conceito: O nosso servidor já está de pé, mas ele não sabe o que fazer quando recebe uma "visita". É aqui que entram as **Rotas** e os **Endpoints**. Um endpoint é o URL final mapeado (como `/perfil` ou `/inicio`). Para criarmos isso, o Flask usa o conceito de ***decorator*** – representado pelo símbolo `@`.

Um *decorator* `@app.route()` é basicamente um "porteiro" programado para interceptar e escutar ações e requisições HTTP feitas pelo usuário. Quando o navegador tentar acessar o caminho que nós configuramos, o *decorator* detecta o acesso e executa a função Python que estiver imediatamente abaixo dele. É a fusão perfeita entre a rede e a nossa lógica de programação.

B. Visualizando:



C. Exemplo de Código:

```python
from flask import Flask
app = Flask(__name__)

# Definimos o padrão de URL (rota) que o navegador vai acessar
@app.route('/inicio')
def pagina_inicial():
    # A função é ativada pela rota. O return agora devolve dados ao navegador!
    return "<h1>Olá, Mundo! Este é o back-end respondendo.</h1>"

if __name__ == '__main__':
    app.run(debug=True)
```

D. Explicação linha por linha:

- `@app.route('/inicio')`: Nós definimos que, se o usuário digitar `/inicio` na barra de endereços, esta será a rota interceptada.
- `def pagina_inicial():`: Criamos a lógica (conhecida como *view* no mercado) atrelada à rota acima. O nome da função pode ser qualquer um que faça sentido para o projeto.
- `return "<h1>...</h1>"`: Nós modificamos o paradigma do `return`. Ao invés de retornar uma variável ou número para dentro do próprio algoritmo, o return injeta essa *String* com *tags* de título direto na comunicação de rede, e o navegador a converte visualmente.

E. Experimente e Responda:

1. Adicione a rota `/inicio` no seu `app.py`. Teste no navegador.
2. Crie uma segunda rota chamada `/contato` que retorne seu email na tela.
3. Se você criar uma função sem um `@app.route` em cima dela, o navegador conseguirá acessá-la?
4. O que ocorre se você tentar acessar `localhost:5000/produtos` sem ter mapeado essa rota?
5. Mude a rota para `@app.route('/')`. O que o caractere `/` sozinho significa na estrutura de pastas da web?

---

3. Retorno e Renderização de HTML (O Desafio Prático)

A. Conceito: Retornar uma *String* (texto comum) na nossa função funciona, mas nenhum sistema real do mercado – e nem a nossa SA – é feito de textos puros. Nós precisamos de interfaces ricas, botões, tabelas e estilização. Para isso, o Flask promove uma transição técnica: nós usamos a função importável `render_template`.

Ela instrui o servidor a buscar um arquivo HTML inteiro e prepará-lo para envio. Contudo, há uma regra arquitetural estrita imposta pelo Flask: por padrão, todos os arquivos `.html` devem estar obrigatoriamente agrupados **dentro de uma pasta na raiz** do projeto chamada `templates`. É a organização da casa.

B. Visualizando:

C. Exemplo de Código:

```python
# Adicionamos a ferramenta render_template na nossa importação inicial
from flask import Flask, render_template

app = Flask(__name__)

# Criamos a rota para o desafio prático
@app.route('/meusite')
def apresentar_site():
    # Encontra o HTML na pasta 'templates', processa e devolve a página estruturada
    return render_template('index.html')

if __name__ == '__main__':
    app.run(debug=True)
```

D. Explicação linha por linha:

- `from flask import render_template`: Importamos a habilidade de renderizar arquivos (unindo o back ao front).
- `def apresentar_site():`: Função conectada ao endpoint `/meusite`.
- `return render_template('index.html')`: Em vez de enviar uma frase curta, o Flask vasculha a pasta oculta `templates`, abre o seu `index.html` e "empurra" o código inteiro estruturado pela porta 5000 para o navegador ler e desenhar o visual completo.

E. Exercícios Rápidos:

1. Crie a pasta chamada exatamente `templates` no mesmo local onde está o seu arquivo `app.py`.
2. Mova o arquivo HTML que você criou na aula anterior de *Front-end* para dentro da pasta `templates`.
3. Importe o `render_template` no topo do seu script.
4. Modifique o return de uma das suas rotas para renderizar o seu HTML.
5. Se você nomear a pasta como "htmls" ou "paginas", o Flask encontrará o arquivo automaticamente? Faça o teste.

FAQ - Dúvidas Reais do Mercado

1. Pergunta: E se eu tentar rodar e aparecer a mensagem "Port 5000 is already in use"?

Resposta: Isso significa que outro serviço no seu sistema operacional já está usando essa "estrada". O host está ocupado. Você pode contornar isso dizendo ao Flask para usar a porta seguinte: `app.run(port=5001, debug=True)`.

2. Pergunta: Por que o arquivo principal costuma se chamar `app.py` ou `main.py`?

Resposta: É uma convenção entre programadores para facilitar a rastreabilidade do código. Qualquer pessoa da equipe que pegar o projeto saberá que ali está a ignição do sistema.

3. Pergunta: O Flask entende CSS e Imagens, ou só HTML?

Resposta: Sim, ele entende! Porém, enquanto os HTMLs ficam na pasta `templates`, os recursos estáticos (CSS, imagens, fontes) devem ir obrigatoriamente para outra pasta raiz chamada `static`.

4. Pergunta: O `debug=True` é seguro para sistemas no ar (produção)?

Resposta: Jamais! Em ambiente de desenvolvimento local ele nos salva apontando o local do erro com exatidão. Se for para produção (deploy), deixamos o modo desligado, caso contrário exporemos nosso código-fonte para usuários comuns (ou hackers).

5. Pergunta: Se um projeto Django é mais completo (full-stack), por que a indústria ainda procura devs Flask?

Resposta: Justamente pela leveza e flexibilidade de construir apenas APIs ou serviços independentes rápidos (microsserviços). O Flask não força o desenvolvedor a seguir uma arquitetura engessada, promovendo integração fácil entre plataformas diferentes.

Exercícios de Fixação

Com base no conteúdo da **Aula 19 - Framework Web (Flask) e Rotas**, elaborei uma trilha de 10 exercícios progressivos. Eles foram desenhados para levar você do zero — a instalação e o "olá mundo" — até a integração completa da sua Situação de Aprendizagem (SA).

1. O Despertar do Servidor
Descrição: Configure o ambiente e inicie um servidor Flask básico. Certifique-se de que ele só rode se o arquivo for executado diretamente e com o modo de depuração ativado.
Entrega: Print do terminal exibindo a mensagem "Running on http://127.0.0.1:5000".

2. Rota Ping-Pong
Descrição: Crie um endpoint chamado `/ping`. O objetivo é entender como o decorator `@app.route` intercepta a requisição e retorna uma resposta simples.
Entrega: Captura de tela do navegador exibindo o texto "Pong!" ao acessar o endereço.

3. Identidade no Back-end
Descrição: Configure a rota raiz (`/`) para que, ao ser acessada, exiba uma String contendo seu Nome, sua Turma do SENAI e o tema escolhido para sua SA.
Entrega: Execução comprovada via localhost no navegador.

4. O Menu de Navegação (Múltiplas Rotas)
Descrição: Desenvolva um script que possua três rotas simultâneas: `/login`, `/cadastro` e `/sobre`. Cada uma deve retornar um texto orientando o usuário sobre em qual seção ele está.
Entrega: O código do arquivo `app.py` completo contendo os três decorators.

5. Arquiteto de Pastas
Descrição: Aplique os princípios de organização (CS2) criando a estrutura obrigatória de pastas na raiz do seu projeto: `templates` (para HTML) e `static` (para CSS/Imagens).
Entrega: Print da árvore de arquivos no VS Code estruturada corretamente.

6. A Primeira Renderização
Descrição: Use a função `render_template` para que a rota `/teste` não retorne apenas um texto, mas sim um arquivo HTML real chamado `teste.html` guardado na pasta correta.
Entrega: O código Python utilizando o `import` e o comando `render_template` corretamente.

7. Explorando o Traceback (Erro Proposital)
Descrição: Com o `debug=True` ativo, provoque um erro de divisão por zero dentro de uma função de rota. Isso serve para aprender a ler o relatório de erros do Werkzeug.

Entrega: Print do navegador exibindo o erro detalhado e uma breve descrição de onde o console indica que a lógica quebrou.

8. Mudança de "Estrada" (Portas Customizadas)
Descrição: Altere as configurações de inicialização para que o servidor suba na porta 8080 em vez da 5000, simulando a resolução de conflitos de rede.
Entrega: Captura do navegador acessando o site via `localhost:8080`.

9. A Rota Perdida (Relatório 404)
Descrição: Tente acessar uma rota inexistente (ex: `/admin_secreto`) e analise a resposta do servidor.
Entrega: Um pequeno relatório (texto) explicando o que o código de erro 404 significa no contexto de rotas mapeadas.

10. Integração Final: A Ponte da SA
Descrição: O desafio mestre! Pegue todo o front-end (HTML/CSS) da sua Situação de Aprendizagem, organize nas pastas `templates` e `static`, e crie uma rota `/sistema` que renderize sua aplicação funcional.
Entrega: Demonstração do projeto front-end completo sendo servido pelo back-end Python na porta 5000.

Entrega: Estes exercícios **NÃO** serão entregues no AVA, você deve apresentar ao professor durante esta aula, toda estrutura de pastas e o servidor funcionando. **APRESENTAÇÃO INDIVIDUAL.**

Fechamento

Conexão com a Situação de Aprendizagem:
Hoje nós construímos a grande "ponte" do nosso projeto final ("Da Ideia ao Código: Construção de Aplicações Funcionais"). O seu banco de dados Python não podia falar diretamente com a tela HTML do usuário. Agora, com a aplicação Flask estruturada, nós já temos o maestro que vai ligar a inteligência, as funções e o armazenamento aos botões e interfaces que o usuário manipula no Front-end (C7).

Próxima Aula:
Com o nosso arquivo HTML renderizando pelo servidor, chegou a hora de dar vida a ele! Na próxima aula, aprenderemos a transitar de "arquivos HTML estáticos e mortos" para interfaces que recebem e desenham dados reais do sistema no momento em que a página carrega, explorando o poder das variáveis dinâmicas no Back-end com *Jinja2*. Preparem-se!